ITA201 – Toán cho học máy

ASSIGNMENT: SMARTEDU AI

Tên đề tài

Xây dựng hệ thống phân tích học tập và tối ưu hỗ trợ sinh viên dựa trên dữ liệu, đồ thị và toán cho học máy

Bối cảnh

Một trung tâm đào tạo có dữ liệu học tập của sinh viên trong một lớp hoặc nhiều lớp, bao gồm:

- Thông tin chuyên cần.
- Điểm quiz.
- Điểm lab.
- Điểm assignment.
- Số giờ tự học.
- Số bài nộp muộn.
- Số lần xem lại tài liệu/video.
- Mức độ tương tác học tập.
- Kết quả cuối kỳ hoặc nhãn nguy cơ học tập.

Trung tâm cần một hệ thống hỗ trợ giảng viên trả lời các câu hỏi:

- Sinh viên nào có nguy cơ chưa hoàn thành môn học?
- Đặc trưng nào có khả năng phân tách nhóm nguy cơ tốt hơn?
- Dữ liệu có nhiều feature dư thừa hoặc tương quan không?
- Có những nhóm sinh viên liên kết hoặc hỗ trợ học tập với nhau không?
- Hành vi học tập có thể chuyển đổi qua các trạng thái như thế nào?

- Với nguồn lực hữu hạn, nên phân công mentor hoặc buổi phụ đạo cho sinh viên nào?
- Làm sao tối ưu số sinh viên nhận được hỗ trợ mà vẫn không vượt năng lực mentor và thời gian hỗ trợ?

Nhóm sinh viên xây dựng một chương trình Python sử dụng các kiến thức toán học đã học để phân tích dữ liệu, dự báo rủi ro đơn giản và đề xuất phương án hỗ trợ tối ưu.

Sản phẩm cuối cùng

Mỗi nhóm nộp một sản phẩm gồm:

- Source code Python có thể chạy lại.
- Một notebook Jupyter hoặc Google Colab trình bày đầy đủ quy trình.
- File dữ liệu CSV tự tạo hoặc thu thập từ nguồn công khai phù hợp.
- Báo cáo PDF.
- Slide thuyết trình/bảo vệ.
- Video demo ngắn nếu giảng viên yêu cầu.
- File README.md hướng dẫn cài đặt, chạy chương trình và mô tả cấu trúc thư mục.

Dự án **không yêu cầu** xây dựng Deep Learning phức tạp. Trọng tâm là chứng minh sinh viên hiểu và áp dụng được công cụ toán học vào bài toán AI/ML.

YÊU CẦU DỮ LIỆU

Phương án dữ liệu khuyến nghị

Mỗi nhóm sử dụng một dataset gồm tối thiểu:

- 100 bản ghi nếu dùng dữ liệu công khai.
- Hoặc 40–60 bản ghi nếu tự tạo dữ liệu mô phỏng có giải thích rõ cách tạo.
- Ít nhất 6 feature dạng số.
- Ít nhất 2 feature phân loại hoặc rời rạc.
- Một nhãn mục tiêu để phân loại, ví dụ nguy_co_cao/nguy_co_thap, hoan_thanh/chua_hoan_thanh, hoặc dat/chua_dat.
- Một trường định danh mẫu, ví dụ student_id.
- Dữ liệu quan hệ cho đồ thị, ví dụ file interactions.csv, student_groups.csv hoặc student_support_network.csv.

Cấu trúc dữ liệu gợi ý:

```
data/
├── students.csv
├── interactions.csv
├── mentors.csv
└── support_capacity.csv
```

File students.csv

Ví dụ các cột:

Cột	Kiểu dữ liệu	Ý nghĩa
student_id	Chuỗi hoặc số	Mã sinh viên, chỉ dùng định danh; không dùng làm feature train
attendance_rate	Số thực	Tỷ lệ chuyên cần, từ 0 đến 1
quiz_avg	Số thực	Điểm quiz trung bình
lab_avg	Số thực	Điểm lab trung bình
assignment_score	Số thực	Điểm Assignment
self_study_hours	Số thực	Số giờ tự học mỗi tuần
late_submissions	Số nguyên	Số bài nộp muộn
video_views	Số nguyên	Số lần xem lại học liệu
participation_level	Phân loại	Thấp, trung bình, cao
risk_label	Nhãn	high_risk hoặc low_risk

Nhóm có thể đổi ngữ cảnh sang khách hàng, giao dịch, đơn hàng, bệnh viện, sản phẩm hoặc người dùng ứng dụng. Tuy nhiên, phải bảo đảm vẫn thực hiện được đầy đủ yêu cầu đại số tuyến tính, PCA, xác suất, entropy, đồ thị, Markov và tối ưu hóa.

File interactions.csv

Ví dụ:

source_student	target_student	interaction_count	relation_type
SV001	SV002	8	study_together
SV001	SV005	3	discuss_assignment
SV003	SV004	5	peer_support

Dòng dữ liệu biểu diễn một cạnh trong đồ thị. interaction_count có thể được dùng làm trọng số cạnh.

File mentors.csv

Ví dụ:

mentor_id	specialty	max_students	available_hours
M01	Python	5	6
M02	Data analysis	4	5
M03	Machine learning	3	4

File support_capacity.csv

Ví dụ:

support_type	capacity	cost_per_student	priority_score
Online mentoring	12	1	5
Lab tutoring	8	2	8
One-to-one support	4	4	10

Phần A. Chuẩn bị và phân tích dữ liệu

A.1. Làm sạch dữ liệu

Trong AI/ML, một dataset được biểu diễn dưới dạng ma trận dữ liệu. Việc kiểm tra và xử lý dữ liệu thiếu, dữ liệu nhiễu hoặc các dòng trùng lặp là bước bắt buộc trước khi đưa vào mô hình toán học

Yêu cầu thực hiện:

1. Dùng thư viện pandas đọc các file CSV (students.csv, interactions.csv,...).
2. Kiểm tra cấu trúc: in số dòng, số cột, kiểm tra kiểu dữ liệu.
3. Kiểm tra và xử lý giá trị thiếu.
4. Kiểm tra dòng trùng lặp.
5. Kiểm tra phạm vi hợp lý của dữ liệu.
6. Mã hóa các đặc trưng phân loại.

Lưu ý quan trọng: Tách riêng cột student_id làm định danh, **không bao giờ** dùng student_id làm feature huấn luyện mô hình.

Báo cáo trong Notebook:

Nêu rõ dataset ban đầu có bao nhiêu lỗi, cách xử lý lỗi, lý do chọn phương pháp xử lý đó, số lượng bản ghi trước và sau khi làm sạch.

A.2. Biểu diễn dữ liệu bằng ma trận và Phép nhân $y = Xw$

Một bảng dữ liệu m sinh viên với n đặc trưng được biểu diễn bằng ma trận $X \in \mathbb{R}^{m \times n}$ và vector nhãn $y \in \mathbb{R}^{m}$.

Phép nhân ma trận–vector Ax hoặc Xw chính là **tổ hợp tuyến tính của các cột trong ma trận X**, đây là nền tảng của mọi mô hình tuyến tính trong ML.

Yêu cầu thực hiện:

1. Tách ma trận đặc trưng X và vector nhãn y.
2. In kích thước của X và. Giải thích rõ mỗi dòng, mỗi cột.
3. Lấy ra tối thiểu 3 mẫu dữ liệu đầu tiên để minh họa bằng ma trận con.
4. Thực hiện ít nhất một phép nhân ma trận–vector có ý nghĩa.
5. Tính điểm nguy cơ.

Ví dụ, tạo vector trọng số giả định:

$$w = \begin{pmatrix} 0.25 \\ 0.30 \\ 0.25 \\ -0.20 \end{pmatrix}$$

và tính điểm nguy cơ đơn giản: $s = Xw$

Nhóm không cần chứng minh đây là mô hình dự báo tốt mà là chứng minh nhóm hiểu ý nghĩa của phép nhân ma trận–vector trong mô hình học máy.

A.3. Norm và khoảng cách

Norm là hàm đo độ lớn của vector. Norm L_2 (Euclidean) đo khoảng cách đường thẳng ("đường chim bay"), trong khi Norm L_1 (Manhattan) đo tổng độ lệch tuyệt đối theo từng trục ("đường đi bộ trong thành phố"). Chuẩn hóa feature (StandardScaler) giúp đưa các thuộc tính có đơn vị đo khác nhau về cùng một thang đo công bằng.

Công thức:

$$D_{L_2}(x, z) = \sqrt{\sum_{i=1}^{n} (x_i - z_i)^2}$$

$$D_{L_1}(x, z) = \sum_{i=1}^{n} |x_i - z_i|$$

Yêu cầu thực hiện:

1. Chọn 2 sinh viên cụ thể trong dataset (ví dụ: Sinh viên A và Sinh viên B). Chuẩn hóa ma trận đặc trưng X. tính:
   - Khoảng cách Euclidean, tương ứng norm L_2.
   - Khoảng cách Manhattan, tương ứng norm L_1.
2. Phân tích kết quả:
   - Giải thích hai sinh viên giống/khác nhau ở feature nào?
   - Giải thích vì sao hai khoảng cách có thể khác nhau?
   - Khi nào cần chuẩn hóa feature trước khi đo khoảng cách.
   - Nêu ứng dụng của khoảng cách trong thuật toán KNN (tìm k sinh viên có hành vi học tập gần nhất), K-Means clustering hoặc phát hiện điểm bất thường.

A.4. Tổ hợp và không gian lựa chọn

Lý thuyết tổ hợp giúp đếm số lượng cấu hình/phương án và đánh giá mức độ phức tạp trước khi thử nghiệm mô hình.

Phép nhân dùng khi công việc gồm nhiều bước liên tiếp, phép cộng dùng khi các nhánh loại trừ lẫn nhau, còn tổ hợp $C(n,k)$ dùng khi chọn k phần tử từ n mà không xét thứ tự.

Nhóm chọn ít nhất một bài toán tổ hợp liên quan dataset.

Ví dụ:

- Dataset có 10 feature và nhóm muốn chọn 3 feature để thử nghiệm.
- Số tập feature có thể chọn là:

$$C(10,3) = \frac{10!}{3!\,7!} = 120$$

Hoặc nhóm có 3 learning rate, 2 batch size và 2 thuật toán, số cấu hình Grid Search là:

$$3 \times 2 \times 2 = 12$$

Yêu cầu thực hiện:

1. Nêu rõ tình huống dùng tổ hợp, chỉnh hợp hoặc nguyên lý nhân.
2. Tính số lượng phương án có thể có.
3. Nhận xét liệu có nên thử toàn bộ hay không.
4. Đề xuất cách thu hẹp không gian tìm kiếm, chẳng hạn chọn feature theo correlation, variance, Information Gain hoặc dùng Random Search.

Phần B. Giảm chiều và đánh giá feature

B.1. Phân tích tương quan và tính dư thừa

Các đặc trưng học tập thường có xu hướng biến thiên cùng nhau (ví dụ: tự học nhiều thì điểm quiz và bài tập cao). Khi hai feature tương quan mạnh, chúng mang thông tin lặp lại (đa cộng tuyến), làm tăng chi phí tính toán và gây mất ổn định cho mô hình tuyến tính.

Yêu cầu thực hiện:

1. Vẽ heatmap tương quan giữa các feature số.
2. Chỉ ra ít nhất 1 cặp feature tương quan đáng chú ý nếu dataset có.
3. Phân tích rủi ro khi đưa nhiều feature dư thừa vào một số mô hình.

B.2. PCA và trị riêng

PCA chuyển đổi hệ trục tọa độ ban đầu sang một hệ trục mới gồm các **Thành phần chính (Principal Components)**.

Về mặt toán học:

- **Vector riêng (Eigenvector)**: Chỉ ra hướng biến thiên của dữ liệu.
- **Trị riêng (Eigenvalue)**: Cho biết độ rộng/mức độ thay đổi của dữ liệu theo hướng đó.
- PCA giữ lại các hướng có trị riêng lớn nhất.

Yêu cầu thực hiện:

1. Chuẩn hóa dữ liệu số, sau đó thực hiện PCA.
2. Báo cáo explained variance ratio của các principal components.
3. Chọn số component sao cho giữ lại tối thiểu 80% phương sai, hoặc giải thích rõ lý do chọn ngưỡng khác.
4. Giảm dữ liệu xuống 2 component để trực quan hóa.
5. Vẽ scatter plot $\mathrm{PC}_1 - \mathrm{PC}_2$, tô màu theo risk_label.

Sinh viên không bắt buộc phải tự hiện thực đầy đủ PCA từ đầu. Có thể dùng sklearn.decomposition.PCA, nhưng phải giải thích:

- Vector riêng biểu diễn các hướng dữ liệu quan trọng.
- Trị riêng cho biết mức độ biến thiên theo từng hướng.
- Component có trị riêng lớn thường được ưu tiên giữ lại.
- PCA tạo feature mới, không đơn giản là xóa bớt feature cũ.

6. Lập bảng đối chiếu ***trước PCA*** (số feature gốc, độ phức tạp) với ***sau PCA*** (số PC giữ lại, % phương sai bảo toàn, khả năng trực quan hóa).

B.3. So sánh mô hình trước và sau PCA

PCA giúp giảm số chiều và tăng tốc huấn luyện, nhưng các Component mới (PC_1 , PC_2) là tổ hợp tuyến tính của các cột cũ nên khó giải thích theo ý nghĩa nghiệp vụ ban đầu.

Cần thực nghiệm kiểm chứng xem PCA có thực sự cải thiện hay làm giảm độ chính xác của mô hình.

Yêu cầu thực hiện:

1. Chọn một mô hình phân loại cơ bản (*ví dụ:* Logistic Regression, KNN, Decision Tree hoặc Naive Bayes).

2. Huấn luyện mô hình trên 2 tập dữ liệu:
   - Tập 1 dùng Feature gốc
   - Tập 2 dùng dữ liệu sau khi PCA.
3. Đánh giá và so sánh bộ chỉ số: Accuracy, Precision, Recall, F1-score và Confusion Matrix.
4. Đưa ra nhận xét khách quan:
   - PCA có làm mô hình tốt hơn không?
   - PCA có làm mô hình chạy nhanh hơn hoặc đơn giản hơn không?
   - Nếu accuracy giảm, nguyên nhân có thể là gì?
   - Nếu khả năng giải thích giảm, có đáng đánh đổi hay không?

Không được mặc định PCA luôn làm độ chính xác tăng. PCA là kỹ thuật giảm chiều và có thể hữu ích hoặc không tùy dữ liệu, mô hình và mục tiêu.

Phần C. Xác suất, Bayes và lý thuyết thông tin

C.1. Tính xác suất có điều kiện

Xác suất có điều kiện

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

đo lường khả năng xảy ra sự kiện A khi đã biết sự kiện B đã xuất hiện.

Cần phân biệt rõ hai chiều điều kiện: $P(Nhãn \mid Feature)$ giúp dự đoán nhãn, còn $P(Feature \mid Nhãn)$ phản ánh đặc điểm của lớp.

Yêu cầu thực hiện:

1. Chọn một feature rời rạc (hoặc nhóm hóa feature số), ví dụ:
   - attendance_group: thấp/cao.
   - late_submission_group: có/không.
   - participation_level: thấp/trung bình/cao.
   - self_study_group: ít/nhiều.
2. Tính ít nhất một xác suất có điều kiện đối lập, ví dụ:

   $P(high_rish \mid attendance_low)$: Trong nhóm sinh viên có chuyên cần thấp, tỷ lệ sinh viên thuộc nhóm nguy cơ cao là bao nhiêu?

   $P(attendance_low \mid high_rish)$: Trong số những sinh viên thuộc nhóm nguy cơ cao, tỷ lệ sinh viên có chuyên cần thấp là bao nhiêu?
3. Giải thích sự khác biệt bản chất giữa hai con số này để chứng minh sự hiểu biết về xác suất điều kiện.

C.2. Phân loại nhãn bằng Bayes

Định lý Bayes cập nhật dự đoán nhãn dựa trên bằng chứng quan sát được:

$$P(Nhãn \mid Feature) = \frac{P(Feature \mid Nhãn) \times P(Nhãn)}{P(Feature)}$$

Mô hình Naive Bayes giả định các đặc trưng độc lập có điều kiện khi đã biết nhãn.

Yêu cầu thực hiện:

1. Chia tập dữ liệu thành Train set và Test set (ví dụ 80/20).
2. Huấn luyện mô hình bằng Gaussian Naive Bayes hoặc Multinomial Naive Bayes tùy dạng dữ liệu.
3. Báo cáo kết quả dự đoán trên tập test.
4. Chọn 1 sinh viên mới và in ra xác suất dự báo (predict_proba).

5. Diễn giải các thành phần Bayes trong báo cáo:
   - Prior probability.
   - Likelihood.
   - Posterior probability.
   - Vai trò của các feature trong dự báo nhãn.

C.3. Entropy và Information Gain

Entropy: $H(S) = -\sum_{i=1}^{k} p_i \log_2(p_i)$ đo độ hỗn loạn/bất định của tập dữ liệu.

Information Gain: $IG(S, A) = H(S) - \sum_{v \in Value(A)} \frac{|S_v|}{|S|} H(S_v)$ đo lượng entropy giảm đi sau khi chia dữ liệu theo đặc trưng A.

Thuật toán Cây quyết định (Decision Tree) ưu tiên chọn feature có Information Gain cao nhất để tạo nút phân tách.

Yêu cầu thực hiện:

1. Tính Entropy ban đầu của nhãn $H(risk_lable)$ trên toàn bộ tập dữ liệu.
2. Chọn tối thiểu 3 feature rời rạc hoặc đã rời rạc hóa.
3. Tính hoặc dùng thư viện để báo cáo Information Gain của từng feature.
4. Xếp hạng feature theo mức độ Information Gain.
5. Nhận xét feature nào giúp phân tách nhãn tốt hơn

Lưu ý: Sinh viên có thể sử dụng mutual_info_classif trong scikit-learn để hỗ trợ đánh giá feature, nhưng phải phân biệt:

- Entropy và Information Gain là nội dung lý thuyết cần trình bày.
- Hàm thư viện chỉ là công cụ hỗ trợ tính toán.
- Kết quả từ thư viện cần được giải thích dựa trên dữ liệu thực tế.

Phần D. Phần D: Đồ thị, BFS/DFS và chuỗi Markov

D.1. Xây dựng đồ thị quan hệ interactions.csv.

Quy ước:

- Đỉnh: sinh viên hoặc đối tượng chính trong bài toán.
- Cạnh: tương tác, hỗ trợ, làm nhóm, trao đổi tài liệu hoặc quan hệ phù hợp với ngữ cảnh.
- Trọng số cạnh: số lần tương tác, số lần hỗ trợ hoặc mức độ tương tác.

Có thể dùng đồ thị vô hướng nếu quan hệ là hai chiều, dùng đồ thị có hướng nếu quan hệ có chiều

Đồ thị $G = (V, E)$ biểu diễn dữ liệu quan hệ, với Đỉnh V là sinh viên và cạnh E là tương tác hỗ trợ học tập.

Đồ thị có thể biểu diễn bằng Ma trận kề (nhanh nhưng tốn bộ nhớ $O(|V|^2)$ hoặc Danh sách kề (tiết kiệm bộ nhớ cho đồ thị thưa).

Yêu cầu thực hiện:

1. Dùng thư viện networkx tạo đồ thị từ file interactions.csv.
2. Báo cáo tổng số đỉnh $|V|$, tổng số cạnh $|E|$, giải thích lý do chọn đồ thị vô hướng hay có hướng.
3. Xuất Danh sách kề (Adjacency List) cho ít nhất 5 đỉnh.
4. Xuất Ma trận kề (Adjacency Matrix) cho một đồ thị con gồm 5–10 đỉnh.

D.2. Duyệt đồ thị BFS, DFS và kiểm tra tính liên thông

- **BFS (Breadth-First Search)**: Duyệt theo từng lớp khoảng cách (dùng Queue / FIFO).
- **DFS (Depth-First Search)**: Đi sâu nhất theo từng nhánh rồi quay lui (dùng Stack / Đệ quy).
- Độ phức tạp của cả hai thuật toán là $O(|V| + |E|)$ khi dùng danh sách kề.

Yêu cầu thực hiện:

1. Chọn 1 đỉnh bắt đầu.
2. Thực hiện thuật toán BFS và in ra thứ tự danh sách các đỉnh được thăm.
3. Thực hiện thuật toán DFS từ cùng đỉnh bắt đầu và in ra thứ tự.
4. So sánh và giải thích lý do thứ tự duyệt của BFS và DFS khác nhau.
5. Kiểm tra tính liên thông:
6. Nếu không liên thông, tìm các thành phần liên thông.

D.3. Ứng dụng chuỗi Markov trong mô hình hoá hành vi học tập

Chuỗi Markov mô hình hóa sự chuyển đổi giữa các trạng thái theo xác suất, trong đó trạng thái tiếp theo chỉ phụ thuộc vào trạng thái hiện tại.

Ma trận xác suất chuyển đổi P có điều kiện bắt buộc: **tổng mỗi hàng phải bằng đúng 1** ($\sum_{j=1}^{n} p_{ij} = 1$).

Yêu cầu thực hiện:

Thiết kế tối thiểu 3 trạng thái học tập hoặc hành vi.

1. Lập Ma trận chuyển đổi với các xác suất hợp lý. Giải thích ý nghĩa của từng trạng thái và giải thích ý nghĩa xác suất chuyển giữa các trạng thái.
2. Chứng minh quy tắc toán học: Cộng tổng từng dòng trong ma trận P để xác nhận đều bằng 1.
3. Chọn một phân phối trạng thái ban đầu π_0. Tính phân phối sau một bước: $\pi_1 = \pi_0 P$
4. Mô phỏng qua $k = 10, 20$ bước ($\pi_k = \pi_0 P^k$) để quan sát sự hội tụ về phân phối dừng (Stationary Distribution) $\pi P = \pi$ đưa ra nhận xét về tỷ lệ sinh viên hoàn thành môn học trong dài hạn.

Đây là yêu cầu nâng cao, có điểm thưởng, nhóm chỉ cần mô phỏng nhiều bước và nhận xét xu hướng nếu chưa giải được bằng đại số.

Phần E. Tối ưu hóa hỗ trợ sinh viên

E.1. Mô hình hóa và giải bài toán Quy hoạch Tuyến tính (Linear Programming)

Bài toán tối ưu hóa gồm Biến quyết định, Hàm mục tiêu (max/min) và các Ràng buộc nguồn lực. Khi các biến nhận giá trị nhị phân 0-1 (được nhận hỗ trợ hay không), bài toán trở thành Quy hoạch nguyên nhị phân (Binary Integer Programming).

Bài toán thực tế

Sinh viên xây dựng bài toán phân bổ mentor hoặc hoạt động hỗ trợ cho sinh viên nguy cơ cao.

Ví dụ:

- Có 12 sinh viên nguy cơ cao.
- Có 3 mentor.
- Mỗi mentor có giới hạn số sinh viên có thể hỗ trợ.
- Mỗi buổi phụ đạo có chi phí.
- Tổng ngân sách hoặc tổng số giờ hỗ trợ là hữu hạn.
- Mỗi sinh viên chỉ được nhận tối đa một hình thức hỗ trợ chính.

Nhóm có thể dùng dự báo của mô hình ở Phần C để tạo priority_score hoặc xác suất nguy cơ cho mỗi sinh viên.

Mục tiêu:

Phân bổ nguồn lực để tổng mức ưu tiên hoặc tổng số sinh viên được hỗ trợ là lớn nhất, nhưng không vượt giới hạn mentor, thời gian và ngân sách.

Mô hình hóa quy hoạch tuyến tính

Ví dụ: đặt

$$x_i = \begin{cases} 1, & nếu\ \text{sinh}\ viên\ i\ \text{được}\ hỗ\ trợ \\ 0, & nếu\ không \end{cases}$$

Hàm mục tiêu tối đa hóa tổng điểm ưu tiên:

$$max\ Z = \sum_{i=1}^{n} p_i x_i$$

Các ràng buộc:

- Ràng buộc ngân sách:

$$\sum_{i=1}^{n} c_i x_i \leq B$$

Trong đó:

  - c_i là chi phí hỗ trợ sinh viên i.
  - B là tổng ngân sách.

- Ràng buộc số sinh viên tối đa mà mentor nhận:

$$\sum_{i \in M_j} x_i \leq capacity_j$$

Lập trình giải bài toán:

Sử dụng thư viện PuLP trong Python để khai báo LpProblem, giải bài toán và in ra danh sách các sinh viên cụ thể được chọn hỗ trợ cùng giá trị.

Sinh viên không bắt buộc phải viết mô hình tổng quát phức tạp. Có thể giải bài toán nhỏ với 5–10 sinh viên, nhưng phải có đủ:

- Biến quyết định.
- Hàm mục tiêu.
- Ít nhất hai ràng buộc.
- Điều kiện biến nhị phân hoặc không âm.
- Lời giải và diễn giải theo ngữ cảnh.

E.2. Mở rộng bằng luồng cực đại và đồ thị hai phía

Phần này được tính điểm nâng cao.

Bài toán ghép cặp sinh viên với mentor là một Đồ thị hai phía (Bipartite Graph). Có thể chuyển bài toán này thành Bài toán Luồng cực đại (Max Flow) bằng cách thêm đỉnh nguồn s và đỉnh đích t.

Thực hiện mô hình hóa việc ghép sinh viên với mentor thành đồ thị hai phía:

Nguồn → Sinh viên → Mentor → Đích

Quy ước:

- Nguồn → sinh viên: dung lượng 1.
- Sinh viên → mentor: có cạnh nếu mentor phù hợp với nhu cầu hỗ trợ.
- Mentor → đích: dung lượng bằng số sinh viên mentor có thể nhận.

Yêu cầu thực hiện:

1. Vẽ/mô tả sơ đồ mạng luồng.
2. Chỉ ra nguồn, đích, cạnh và dung lượng.
3. Tìm số sinh viên tối đa được ghép với mentor.
4. Giải thích tại sao giá trị Max Flow chính bằng số lượng sinh viên tối đa được ghép cặp thành công với mentor *(ghép cặp hợp lệ).*
5. Nhận diện Nút thắt cổ chai (Bottleneck / Min-Cut): Chỉ ra mentor nào hoặc hình thức hỗ trợ nào đang là nút thắt *(bị quá tải/thiếu sức chứa khiến không thể hỗ trợ thêm sinh viên).*

Phần F. Yêu cầu kỹ thuật

F.1. Công nghệ tối thiểu

- Python 3.
- Jupyter Notebook hoặc Google Colab.
- pandas.
- numpy.
- matplotlib hoặc seaborn.
- scikit-learn.
- networkx.
- PuLP hoặc scipy.optimize.

F.2. Gợi ý cấu trúc thư mục

```
smartedu-ai/
├── data/
│   ├── students.csv
│   ├── interactions.csv
│   ├── mentors.csv
│   └── support_capacity.csv
├── notebooks/
│   └── smartedu_assignment.ipynb
├── src/
│   ├── data_processing.py
│   ├── linear_algebra.py
│   ├── pca_analysis.py
│   ├── probability_entropy.py
│   ├── graph_markov.py
│   └── optimization.py
├── outputs/
│   ├── figures/
│   ├── tables/
│   └── model_results/
├── report/
│   └── report.pdf
├── slides/
│   └── presentation.pdf
├── requirements.txt
└── README.md
```

Nhóm có thể chỉ dùng notebook nếu thời lượng giới hạn, nhưng code cần được phân chia rõ theo từng phần yêu cầu và có thể chạy lại từ đầu.

Phần G. Mốc đánh giá tiến độ

Đánh giá giai đoạn 1

Thời điểm đề xuất: sau khi hoàn thành Bài 3 hoặc Bài 4.

Nhóm nộp:

- Tên đề tài và thành viên.
- Mô tả bài toán.
- Nguồn dữ liệu hoặc cách tạo dữ liệu.
- Data dictionary.
- Phân tích dữ liệu ban đầu.
- Ma trận feature và nhãn.
- Phép toán ma trận–vector minh họa.
- Tính norm/khoảng cách.
- Phân tích tổ hợp không gian chọn feature hoặc cấu hình.
- PCA và biểu đồ PC1–PC2.
- Kế hoạch thực hiện giai đoạn 2.

Mục tiêu của mốc này là xác nhận nhóm có dữ liệu phù hợp và đã vận dụng được kiến thức Bài 1–4.

Đánh giá giai đoạn 2

Thời điểm đề xuất: sau khi hoàn thành Bài 7.

Nhóm nộp:

- Bản cập nhật notebook/source code.
- Mô hình Naive Bayes hoặc mô hình baseline.
- Xác suất có điều kiện.
- Entropy và Information Gain.

- Đồ thị quan hệ và kết quả BFS/DFS.
- Kết quả kiểm tra thành phần liên thông.
- Ma trận Markov, kết quả mô phỏng chuyển trạng thái.
- Phát biểu sơ bộ bài toán tối ưu.
- Dự kiến sản phẩm demo và phân công hoàn thiện.

Hết.